FootyEdge AI — Complete PDF Generation Package

PDF 1: FOOTYEDGE_AI_TECHNICAL_SPEC.pdf

```markdown
# FOOTYEDGE AI - COMPLETE TECHNICAL SPECIFICATION
## Version 1.0 | Zero-Cost Enterprise Edition

---

## EXECUTIVE SUMMARY

FootyEdge AI is a zero-cost, production-ready football betting intelligence platform that uses AI agents to predict match outcomes, calculate Expected Value (EV), and provide optimal betting strategies. The system is designed to run entirely on free tiers of cloud services with no monthly costs.

### Core Capabilities
- Match outcome prediction (Win/Draw/Loss) with probabilities
- Expected Goals (xG) calculation
- Value bet detection using Expected Value (EV)
- Kelly Criterion bet sizing optimization
- Multi-agent AI architecture
- Telegram bot interface
- React dashboard with real-time updates
- REST API for third-party integration

### Technology Stack (All Free)
- **Backend**: FastAPI (Python 3.11)
- **Database**: Supabase (PostgreSQL + Realtime)
- **AI/ML**: Custom statistical models + Agent.ai
- **Bot**: python-telegram-bot
- **Frontend**: React + Tailwind CSS
- **Deployment**: Render (API + Bot), Vercel (Dashboard)
- **CI/CD**: GitHub Actions
- **Version Control**: GitHub

---

## SYSTEM ARCHITECTURE

### High-Level Architecture
```

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                USER INTERFACES                                                │
├──────────────────────────────────────────────┬──────────────────────────────────────────────┬──────────────────────────────────────────────┤
│ Telegram Bot                                 │ React Dashboard                              │ REST API Clients                             │
└──────────────────────────────────────────────┴──────────────────────────────────────────────┴──────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                              FOOTYEDGE AI CORE                                              │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                              AGENT ORCHESTRATOR                                             │
│                                Coordinates all AI agents in the workflow                                    │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│                                              │                                              │
┌──────────────────────────────────────────────┬──────────────────────────────────────────────┬──────────────────────────────────────────────┬──────────────────────────────────────────────┐
│ Data Agent                                   │ Team Agent                                   │ Goal Agent                                   │ Bet Agent                                    │
│ Ingests data                                 │ Calculates                                   │ Predicts                                     │ Finds value                                  │
│ from APIs                                    │ team ratings                                 │ scorelines                                   │ bets + Kelly                                 │
└──────────────────────────────────────────────┴──────────────────────────────────────────────┴──────────────────────────────────────────────┴──────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                              DATA STORAGE LAYER                                             │
```

```
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│  Supabase PostgreSQL                │  Redis Cache (optional)          │
│  - teams                            │  - API responses                 │
│  - matches                          │  - Prediction cache              │
│  - predictions                      │                                  │
│  - value_bets                       │                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Agent Workflow

```yaml
Agent Workflow:
  1. Data Ingestion Agent:
     - Fetch match data from Football-Data.org
     - Scrape historical results from free sources
     - Cache responses to respect API limits

  2. Team Strength Agent:
     - Calculate Elo ratings with Bayesian update
     - Weight recent form (exponential decay)
     - Apply home advantage factor (+50 Elo)
     - Track attack/defense strength separately

  3. Goal Distribution Agent:
     - Use Poisson distribution for goal modeling
     - λ_home = home_win_probability × 2.5
     - λ_away = away_win_probability × 2.0
     - Generate score matrix for all outcomes

  4. Betting Strategist Agent:
     - Calculate Expected Value for each market
     - Apply Kelly Criterion for bet sizing
     - Filter for EV > 5%
     - Recommend stake percentage

  5. Insight Agent:
     - Generate human-readable explanations
     - Identify key factors influencing prediction
     - Highlight contrarian angles
```

COMPLETE API SPECIFICATION

Base URL

```
https://footyedge-api.onrender.com
```

Endpoints

1. GET /

Description: Health check and system information

Response:

```json
{
  "name": "FootyEdge AI",
  "version": "1.0.0",
  "status": "running",
  "cost": "$0/month",
  "features": [
    "match_prediction",
    "value_bet_detection",
    "kelly_criterion",
    "telegram_bot"
  ]
}
```

2. POST /predict

Description: Get match prediction and value bets

Request:

```json
{
  "home_team": "Manchester City",
  "away_team": "Arsenal"
```

```
}
```

Response:

```json
{
  "home_team": "Manchester City",
  "away_team": "Arsenal",
  "home_win": 0.52,
  "draw": 0.25,
  "away_win": 0.23,
  "home_xg": 1.85,
  "away_xg": 1.20,
  "best_bet": "Manchester City to win @ 1.85",
  "ev": 0.123,
  "kelly_percentage": 0.08,
  "recommended_stake": "2.0% of bankroll",
  "confidence_score": 0.85,
  "key_factors": [
    "Manchester City strong home form: 5 wins in last 6",
    "Arsenal missing key defender (injury)",
    "Historical: City won 3 of last 4 meetings"
  ]
}
```

3. GET /recent-predictions

Description: Get recent predictions from database

Query Parameters:

· limit (optional, default=10): Number of predictions to return

Response:

```json
{
  "predictions": [
    {
      "id": 1,
      "home_team": "Manchester City",
      "away_team": "Arsenal",
```

```
      "home_prob": 0.52,
      "draw_prob": 0.25,
      "away_prob": 0.23,
      "home_xg": 1.85,
      "away_xg": 1.20,
      "created_at": "2024-01-15T14:30:00Z"
    }
  ],
  "total": 1
}
```

4. GET /value-bets

Description: Get all active value bets for today

Response:

```json
{
  "value_bets": [
    {
      "match": "Manchester City vs Arsenal",
      "market": "home_win",
      "selection": "Manchester City to win",
      "odds": 1.85,
      "our_probability": 0.52,
      "ev": 0.123,
      "kelly": 0.08,
      "recommended_stake": "2.0%"
    },
    {
      "match": "Liverpool vs Chelsea",
      "market": "draw",
      "selection": "Draw",
      "odds": 3.40,
      "our_probability": 0.32,
      "ev": 0.088,
      "kelly": 0.04,
      "recommended_stake": "1.0%"
    }
  ]
}
```

5. GET /teams/{team_name}

Description: Get detailed team statistics

Response:

```json
{
  "team_name": "Manchester City",
  "elo_rating": 1680,
  "attack_strength": 1.42,
  "defense_strength": 0.85,
  "home_advantage": 65,
  "form_rating": 0.92,
  "last_5_results": ["W", "W", "D", "W", "W"],
  "top_scorer": "Erling Haaland (14 goals)",
  "avg_goals_scored": 2.3,
  "avg_goals_conceded": 0.8,
  "clean_sheet_percentage": 0.45
}
```

DATABASE SCHEMA (Supabase)

Complete SQL Schema

```sql
-- ============================================
-- FOOTYEDGE AI DATABASE SCHEMA
-- Run this in Supabase SQL Editor
-- ============================================

-- 1. TEAMS TABLE
CREATE TABLE teams (
    id BIGSERIAL PRIMARY KEY,
    name TEXT UNIQUE NOT NULL,
    country TEXT,
    league TEXT,
    elo_rating FLOAT DEFAULT 1500,
    attack_strength FLOAT DEFAULT 1.0,
    defense_strength FLOAT DEFAULT 1.0,
```

```
    home_advantage FLOAT DEFAULT 50,
    form_rating FLOAT DEFAULT 0.5,
    total_matches INT DEFAULT 0,
    wins INT DEFAULT 0,
    draws INT DEFAULT 0,
    losses INT DEFAULT 0,
    goals_scored INT DEFAULT 0,
    goals_conceded INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- 2. MATCHES TABLE
CREATE TABLE matches (
    id BIGSERIAL PRIMARY KEY,
    home_team_id BIGINT REFERENCES teams(id),
    away_team_id BIGINT REFERENCES teams(id),
    match_date TIMESTAMP NOT NULL,
    league VARCHAR(50),
    season VARCHAR(20),
    home_goals INT,
    away_goals INT,
    home_xg FLOAT,
    away_xg FLOAT,
    home_possession FLOAT,
    away_possession FLOAT,
    home_shots INT,
    away_shots INT,
    home_shots_on_target INT,
    away_shots_on_target INT,
    home_corners INT,
    away_corners INT,
    home_yellow_cards INT,
    away_yellow_cards INT,
    home_red_cards INT,
    away_red_cards INT,
    weather VARCHAR(50),
    temperature FLOAT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 3. PREDICTIONS TABLE
CREATE TABLE predictions (
    id BIGSERIAL PRIMARY KEY,
```

```
    match_id BIGINT REFERENCES matches(id),
    home_team VARCHAR(100),
    away_team VARCHAR(100),
    home_prob FLOAT,
    draw_prob FLOAT,
    away_prob FLOAT,
    home_xg FLOAT,
    away_xg FLOAT,
    confidence FLOAT,
    best_bet_market VARCHAR(50),
    best_bet_selection VARCHAR(100),
    best_bet_odds FLOAT,
    best_bet_ev FLOAT,
    best_bet_kelly FLOAT,
    model_version VARCHAR(20),
    actual_result VARCHAR(10),
    prediction_error FLOAT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 4. VALUE_BETS TABLE
CREATE TABLE value_bets (
    id BIGSERIAL PRIMARY KEY,
    match_id BIGINT REFERENCES matches(id),
    home_team VARCHAR(100),
    away_team VARCHAR(100),
    market VARCHAR(50),
    selection VARCHAR(100),
    odds FLOAT,
    our_probability FLOAT,
    implied_probability FLOAT,
    ev FLOAT,
    kelly_percentage FLOAT,
    recommended_stake_percentage FLOAT,
    recommended_stake_amount FLOAT,
    bankroll_used FLOAT,
    status VARCHAR(20) DEFAULT 'active',
    settled BOOLEAN DEFAULT FALSE,
    actual_win BOOLEAN,
    profit_loss FLOAT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 5. TEAM_RATINGS_HISTORY (Time-series)
```

```
CREATE TABLE team_ratings_history (
    id BIGSERIAL PRIMARY KEY,
    team_id BIGINT REFERENCES teams(id),
    rating_date DATE NOT NULL,
    elo_rating FLOAT,
    attack_strength FLOAT,
    defense_strength FLOAT,
    form_rating FLOAT,
    position_in_league INT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 6. USER_BETS (for tracking user bets)
CREATE TABLE user_bets (
    id BIGSERIAL PRIMARY KEY,
    user_id VARCHAR(100),
    match_id BIGINT REFERENCES matches(id),
    market VARCHAR(50),
    selection VARCHAR(100),
    odds FLOAT,
    stake FLOAT,
    potential_win FLOAT,
    value_bet_id BIGINT REFERENCES value_bets(id),
    actual_result VARCHAR(10),
    profit_loss FLOAT,
    settled_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 7. AGENT_LOGS (for debugging)
CREATE TABLE agent_logs (
    id BIGSERIAL PRIMARY KEY,
    agent_name VARCHAR(50),
    action VARCHAR(100),
    input_data JSONB,
    output_data JSONB,
    execution_time_ms INT,
    success BOOLEAN,
    error_message TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- ==========================================
-- INDEXES FOR PERFORMANCE
```

```
-- ============================================

CREATE INDEX idx_predictions_created ON predictions(created_at DESC);
CREATE INDEX idx_predictions_match ON predictions(match_id);
CREATE INDEX idx_matches_date ON matches(match_date);
CREATE INDEX idx_matches_teams ON matches(home_team_id, away_team_id);
CREATE INDEX idx_value_bets_active ON value_bets(status, created_at);
CREATE INDEX idx_team_ratings_date ON team_ratings_history(rating_date DESC);
CREATE INDEX idx_agent_logs_created ON agent_logs(created_at DESC);

-- ============================================
-- ENABLE ROW LEVEL SECURITY
-- ============================================

ALTER TABLE teams ENABLE ROW LEVEL SECURITY;
ALTER TABLE matches ENABLE ROW LEVEL SECURITY;
ALTER TABLE predictions ENABLE ROW LEVEL SECURITY;
ALTER TABLE value_bets ENABLE ROW LEVEL SECURITY;
ALTER TABLE team_ratings_history ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_bets ENABLE ROW LEVEL SECURITY;
ALTER TABLE agent_logs ENABLE ROW LEVEL SECURITY;

-- Public read access
CREATE POLICY "Enable read access for all users" ON teams FOR SELECT USING (true);
CREATE POLICY "Enable read access for all users" ON matches FOR SELECT USING (true);
CREATE POLICY "Enable read access for all users" ON predictions FOR SELECT USING
(true);
CREATE POLICY "Enable read access for all users" ON value_bets FOR SELECT USING
(true);
CREATE POLICY "Enable read access for all users" ON team_ratings_history FOR SELECT
USING (true);

-- Authenticated write access
CREATE POLICY "Enable insert for authenticated users" ON predictions FOR INSERT WITH
CHECK (true);
CREATE POLICY "Enable insert for authenticated users" ON value_bets FOR INSERT WITH
CHECK (true);

-- ============================================
-- ENABLE REALTIME SUBSCRIPTIONS
-- ============================================

ALTER TABLE predictions REPLICA IDENTITY FULL;
ALTER TABLE value_bets REPLICA IDENTITY FULL;
```

```
ALTER PUBLICATION supabase_realtime ADD TABLE predictions;
ALTER PUBLICATION supabase_realtime ADD TABLE value_bets;

-- ============================================
-- FUNCTIONS AND TRIGGERS
-- ============================================

-- Update updated_at timestamp
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
   NEW.updated_at = NOW();
   RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER update_teams_updated_at BEFORE UPDATE ON teams
   FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

-- Calculate team stats after match
CREATE OR REPLACE FUNCTION update_team_stats()
RETURNS TRIGGER AS $$
BEGIN
   -- Update home team stats
   UPDATE teams
   SET
      total_matches = total_matches + 1,
      goals_scored = goals_scored + NEW.home_goals,
      goals_conceded = goals_conceded + NEW.away_goals,
      wins = wins + CASE WHEN NEW.home_goals > NEW.away_goals THEN 1 ELSE 0 END,
      draws = draws + CASE WHEN NEW.home_goals = NEW.away_goals THEN 1 ELSE 0
END,
      losses = losses + CASE WHEN NEW.home_goals < NEW.away_goals THEN 1 ELSE 0
END
   WHERE id = NEW.home_team_id;

   -- Update away team stats
   UPDATE teams
   SET
      total_matches = total_matches + 1,
      goals_scored = goals_scored + NEW.away_goals,
      goals_conceded = goals_conceded + NEW.home_goals,
      wins = wins + CASE WHEN NEW.away_goals > NEW.home_goals THEN 1 ELSE 0 END,
```

```
        draws = draws + CASE WHEN NEW.away_goals = NEW.home_goals THEN 1 ELSE 0
END,
        losses = losses + CASE WHEN NEW.away_goals < NEW.home_goals THEN 1 ELSE 0
END
    WHERE id = NEW.away_team_id;

    RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER after_match_insert AFTER INSERT ON matches
    FOR EACH ROW EXECUTE FUNCTION update_team_stats();
```

COMPLETE CODE FILES

File 1: predictor.py (Core Prediction Engine)

```python
"""
FootyEdge AI - Core Prediction Engine
Zero-cost statistical model for football match prediction
"""

import requests
import numpy as np
from datetime import datetime, timedelta
import os
import json
from typing import Dict, List, Tuple, Optional
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class FootyEdgePredictor:
    """
    Main prediction engine with:
    - Elo rating system with Bayesian update
    - Poisson goal distribution
    - Kelly Criterion bet sizing
    - Value bet detection
```

```
"""

def __init__(self, supabase_url: str = None, supabase_key: str = None,
             football_api_key: str = None):
    """
    Initialize the predictor with API credentials

    Args:
        supabase_url: Supabase project URL
        supabase_key: Supabase anon/public key
        football_api_key: Football-Data.org API key
    """
    self.supabase_url = supabase_url or os.environ.get('SUPABASE_URL')
    self.supabase_key = supabase_key or os.environ.get('SUPABASE_KEY')
    self.football_api_key = football_api_key or os.environ.get('FOOTBALL_API_KEY')

    self.supabase = None
    if self.supabase_url and self.supabase_key:
        try:
            from supabase import create_client
            self.supabase = create_client(self.supabase_url, self.supabase_key)
            logger.info("Supabase connected successfully")
        except Exception as e:
            logger.error(f"Failed to connect to Supabase: {e}")

    # Cache for API responses
    self.cache = {}
    self.cache_ttl = 3600  # 1 hour

    # Team ratings cache
    self.team_ratings = {}

def _get_from_cache(self, key: str) -> Optional[Dict]:
    """Get data from cache if not expired"""
    if key in self.cache:
        data, timestamp = self.cache[key]
        if (datetime.now() - timestamp).seconds < self.cache_ttl:
            return data
    return None

def _save_to_cache(self, key: str, data: Dict):
    """Save data to cache"""
    self.cache[key] = (data, datetime.now())
```

```python
def get_team_matches(self, team_name: str, limit: int = 10) -> List[Dict]:
    """
    Fetch recent matches for a team from Football-Data.org

    Args:
        team_name: Name of the team
        limit: Number of recent matches to fetch

    Returns:
        List of match dictionaries
    """
    cache_key = f"team_matches_{team_name}_{limit}"
    cached = self._get_from_cache(cache_key)
    if cached:
        return cached

    matches = []

    # Try API first
    if self.football_api_key:
        try:
            # Search for team ID first
            team_id = self._get_team_id(team_name)
            if team_id:
                url = f"https://api.football-data.org/v4/teams/{team_id}/matches"
                headers = {'X-Auth-Token': self.football_api_key}
                params = {'limit': limit, 'status': 'FINISHED'}

                response = requests.get(url, headers=headers, params=params, timeout=10)

                if response.status_code == 200:
                    data = response.json()
                    for match in data.get('matches', []):
                        matches.append(self._parse_match_data(match, team_name))
                    logger.info(f"Fetched {len(matches)} matches for {team_name} from API")
        except Exception as e:
            logger.error(f"API fetch failed for {team_name}: {e}")

    # Fallback to mock data if API fails or no matches
    if not matches:
        matches = self._get_mock_data(team_name)
        logger.info(f"Using mock data for {team_name}")

    self._save_to_cache(cache_key, matches)
```

```
    return matches

def _get_team_id(self, team_name: str) -> Optional[int]:
    """Get team ID from Football-Data.org"""
    # Simplified - in production, cache this mapping
    team_mapping = {
        'Manchester City': 65,
        'Arsenal': 57,
        'Liverpool': 64,
        'Chelsea': 61,
        'Manchester United': 66,
        'Tottenham': 73,
        # Add more as needed
    }
    return team_mapping.get(team_name)

def _parse_match_data(self, match: Dict, team_name: str) -> Dict:
    """Parse API match data into standard format"""
    home_team = match.get('homeTeam', {}).get('name', '')
    away_team = match.get('awayTeam', {}).get('name', '')
    is_home = team_name == home_team

    home_score = match.get('score', {}).get('fullTime', {}).get('home', 0)
    away_score = match.get('score', {}).get('fullTime', {}).get('away', 0)

    if is_home:
        goals_scored = home_score
        goals_conceded = away_score
        if home_score > away_score:
            result = 'win'
        elif home_score == away_score:
            result = 'draw'
        else:
            result = 'loss'
    else:
        goals_scored = away_score
        goals_conceded = home_score
        if away_score > home_score:
            result = 'win'
        elif away_score == home_score:
            result = 'draw'
        else:
            result = 'loss'
```

```
        return {
            'opponent': away_team if is_home else home_team,
            'is_home': is_home,
            'goals_scored': goals_scored,
            'goals_conceded': goals_conceded,
            'result': result,
            'date': match.get('utcDate', '')
        }

    def _get_mock_data(self, team_name: str) -> List[Dict]:
        """Generate mock data for testing"""
        # Create realistic mock data based on team name
        mock_records = {
            'Manchester City': [('win', 3, 0), ('win', 2, 1), ('draw', 1, 1), ('win', 4, 0), ('win', 2, 0)],
            'Arsenal': [('win', 2, 1), ('win', 3, 1), ('draw', 1, 1), ('loss', 0, 2), ('win', 2, 0)],
            'Liverpool': [('win', 3, 1), ('draw', 1, 1), ('win', 2, 0), ('loss', 1, 2), ('win', 4, 1)],
            'Chelsea': [('draw', 1, 1), ('loss', 0, 2), ('win', 2, 0), ('draw', 0, 0), ('loss', 1, 2)],
        }

        records = mock_records.get(team_name, [('win', 2, 1), ('draw', 1, 1), ('loss', 0, 1), ('win', 2,
0), ('draw', 1, 1)])

        matches = []
        for i, (result, scored, conceded) in enumerate(records):
            matches.append({
                'opponent': f'Opponent_{i+1}',
                'is_home': i % 2 == 0,
                'goals_scored': scored,
                'goals_conceded': conceded,
                'result': result,
                'date': (datetime.now() - timedelta(days=i*7)).isoformat()
            })

        return matches

    def calculate_elo_rating(self, team_name: str, matches: List[Dict]) -> float:
        """
        Calculate Elo rating with recency weighting

        Args:
            team_name: Name of the team
            matches: List of recent matches

        Returns:
```

```
        Elo rating (1500 = average)
    """
    if team_name in self.team_ratings:
        rating = self.team_ratings[team_name]
    else:
        rating = 1500

    # Sort matches by date (oldest first for proper sequential update)
    sorted_matches = sorted(matches, key=lambda x: x.get('date', ''))

    for i, match in enumerate(sorted_matches):
        # Weight based on recency (exponential decay)
        weight = 0.95 ** (len(sorted_matches) - i - 1)

        # Get opponent rating (simplified - use average)
        opponent_rating = 1500

        # Expected score based on Elo formula
        expected = 1 / (1 + 10 ** ((opponent_rating - rating) / 400))

        # Actual result
        if match['result'] == 'win':
            actual = 1
        elif match['result'] == 'draw':
            actual = 0.5
        else:
            actual = 0

        # Update rating with recency-weighted K-factor
        k_factor = 32 * weight
        rating += k_factor * (actual - expected)

    # Store in cache
    self.team_ratings[team_name] = rating

    return rating

def calculate_form_rating(self, matches: List[Dict]) -> float:
    """
    Calculate recent form rating (0-1 scale)

    Args:
        matches: List of recent matches
```

```
    Returns:
        Form rating between 0 and 1
    """
    if not matches:
        return 0.5

    total_points = 0
    total_weight = 0

    for i, match in enumerate(matches[:10]):  # Last 10 matches
        weight = 0.9 ** i  # Exponential decay

        if match['result'] == 'win':
            points = 3
        elif match['result'] == 'draw':
            points = 1
        else:
            points = 0

        total_points += points * weight
        total_weight += weight

    if total_weight == 0:
        return 0.5

    # Convert to 0-1 scale (max 3 points per match)
    max_points = 3 * total_weight
    form_rating = total_points / max_points

    return form_rating

def calculate_attack_strength(self, matches: List[Dict]) -> float:
    """Calculate attack strength relative to average"""
    if not matches:
        return 1.0

    goals_scored = [m['goals_scored'] for m in matches]
    avg_goals = np.mean(goals_scored) if goals_scored else 0

    # League average is about 1.5 goals per team per match
    return avg_goals / 1.5

def calculate_defense_strength(self, matches: List[Dict]) -> float:
    """Calculate defense strength relative to average"""
```

```
    if not matches:
        return 1.0

    goals_conceded = [m['goals_conceded'] for m in matches]
    avg_conceded = np.mean(goals_conceded) if goals_conceded else 0

    # League average is about 1.5 goals conceded per match
    return avg_conceded / 1.5

def predict_match(self, home_team: str, away_team: str) -> Dict:
    """
    Predict match outcome with probabilities and xG

    Args:
        home_team: Name of home team
        away_team: Name of away team

    Returns:
        Dictionary with prediction results
    """
    # Get historical data
    home_matches = self.get_team_matches(home_team, limit=10)
    away_matches = self.get_team_matches(away_team, limit=10)

    # Calculate ratings
    home_elo = self.calculate_elo_rating(home_team, home_matches)
    away_elo = self.calculate_elo_rating(away_team, away_matches)

    # Form ratings
    home_form = self.calculate_form_rating(home_matches)
    away_form = self.calculate_form_rating(away_matches)

    # Attack/defense strengths
    home_attack = self.calculate_attack_strength(home_matches)
    away_attack = self.calculate_attack_strength(away_matches)
    home_defense = self.calculate_defense_strength(home_matches)
    away_defense = self.calculate_defense_strength(away_matches)

    # Home advantage factor (standard 50 Elo points ≈ 0.4 goals)
    home_advantage = 50

    # Combine factors into final strength
    home_strength = home_elo + home_advantage + (home_form * 100) + (home_attack * 50)
    away_strength = away_elo + (away_form * 100) + (away_attack * 50)
```

```
        # Base probabilities
        total = home_strength + away_strength
        home_prob_raw = home_strength / total
        away_prob_raw = away_strength / total

        # Draw probability (based on league average and team similarity)
        elo_diff = abs(home_elo - away_elo)
        draw_prob = max(0.20, min(0.35, 0.28 - (elo_diff / 1000)))

        # Adjust win probabilities
        home_prob = home_prob_raw * (1 - draw_prob)
        away_prob = away_prob_raw * (1 - draw_prob)

        # Expected goals (using Poisson lambda)
        home_xg = (home_attack / away_defense) * 1.5 * (1 + home_form)
        away_xg = (away_attack / home_defense) * 1.5 * (1 + away_form)

        # Cap realistic ranges
        home_xg = max(0.8, min(2.5, home_xg))
        away_xg = max(0.5, min(2.0, away_xg))

        # Generate score matrix for top outcomes
        score_matrix = self._generate_score_matrix(home_xg, away_xg)

        return {
            'home_team': home_team,
            'away_team': away_team,
            'home_win': round(home_prob, 3),
            'draw': round(draw_prob, 3),
            'away_win': round(away_prob, 3),
            'home_xg': round(home_xg, 2),
            'away_xg': round(away_xg, 2),
            'home_elo': round(home_elo),
            'away_elo': round(away_elo),
            'home_form': round(home_form, 2),
            'away_form': round(away_form, 2),
            'score_matrix': score_matrix,
            'confidence_score': self._calculate_confidence(home_prob, draw_prob, away_prob)
        }

    def _generate_score_matrix(self, home_xg: float, away_xg: float, max_goals: int = 5) ->
List[Dict]:
        """
```

```
    Generate probability matrix for all possible scorelines using Poisson

    Args:
        home_xg: Home team expected goals
        away_xg: Away team expected goals
        max_goals: Maximum goals to consider

    Returns:
        List of score probabilities
    """
    from math import exp, factorial

    score_matrix = []

    for i in range(max_goals + 1):
        for j in range(max_goals + 1):
            # Poisson probability for each team
            p_home = (exp(-home_xg) * (home_xg ** i)) / factorial(i)
            p_away = (exp(-away_xg) * (away_xg ** j)) / factorial(j)
            prob = p_home * p_away

            score_matrix.append({
                'home_goals': i,
                'away_goals': j,
                'probability': round(prob, 4)
            })

    # Normalize
    total_prob = sum(s['probability'] for s in score_matrix)
    for s in score_matrix:
        s['probability'] = round(s['probability'] / total_prob, 4)

    return score_matrix

def _calculate_confidence(self, home_prob: float, draw_prob: float, away_prob: float) -> float:
    """Calculate confidence score based on probability spread"""
    # Higher confidence when one outcome is much more likely
    max_prob = max(home_prob, draw_prob, away_prob)
    second_prob = sorted([home_prob, draw_prob, away_prob])[1]

    confidence = (max_prob - second_prob) * 2
    return round(min(0.95, max(0.5, confidence)), 2)

def kelly_criterion(self, probability: float, odds: float) -> float:
```

```
    """
    Calculate optimal bet size using Kelly Criterion

    f* = (p * b - q) / b
    where:
    p = probability of winning
    b = decimal odds - 1
    q = 1 - p

    Args:
        probability: Estimated probability of winning
        odds: Decimal odds offered

    Returns:
        Optimal bet percentage of bankroll
    """
    b = odds - 1
    p = probability
    q = 1 - p

    if b <= 0 or p <= 0:
        return 0

    kelly = (p * b - q) / b

    # Cap at 25% and floor at 0
    return max(0, min(kelly, 0.25))

def find_value_bets(self, prediction: Dict, bookmaker_odds: Dict) -> List[Dict]:
    """
    Find bets with positive Expected Value

    Args:
        prediction: Prediction dictionary from predict_match()
        bookmaker_odds: Dictionary of odds from bookmaker
            Example: {'home_win': 1.85, 'draw': 3.40, 'away_win': 4.20}

    Returns:
        List of value bets sorted by EV
    """
    value_bets = []

    markets = [
        ('home_win', prediction['home_win'], f"{prediction['home_team']} to win"),
```

```
        ('draw', prediction['draw'], "Draw"),
        ('away_win', prediction['away_win'], f"{prediction['away_team']} to win")
    ]

    for market_key, our_prob, selection in markets:
        odds = bookmaker_odds.get(market_key)
        if odds and our_prob > 0:
            ev = (our_prob * odds) - 1

            if ev > 0.05:  # 5% edge threshold
                kelly = self.kelly_criterion(our_prob, odds)

                value_bets.append({
                    'market': market_key,
                    'selection': selection,
                    'odds': odds,
                    'our_probability': our_prob,
                    'implied_probability': 1 / odds,
                    'ev': round(ev, 3),
                    'kelly_percentage': round(kelly, 3),
                    'recommended_stake_percentage': round(kelly * 0.25, 3),  # Fractional Kelly for
safety
                    'recommendation': 'Strong Buy' if ev > 0.15 else 'Buy' if ev > 0.08 else 'Consider'
                })

    # Sort by EV descending
    value_bets.sort(key=lambda x: x['ev'], reverse=True)

    return value_bets

  def save_prediction(self, prediction: Dict, value_bets: List[Dict] = None) -> bool:
    """
    Save prediction to Supabase database

    Args:
        prediction: Prediction dictionary
        value_bets: List of value bets (optional)

    Returns:
        True if saved successfully
    """
    if not self.supabase:
        logger.warning("No Supabase connection, skipping save")
        return False
```

```
try:
    # Save prediction
    prediction_data = {
        'home_team': prediction['home_team'],
        'away_team': prediction['away_team'],
        'home_prob': prediction['home_win'],
        'draw_prob': prediction['draw'],
        'away_prob': prediction['away_win'],
        'home_xg': prediction['home_xg'],
        'away_xg': prediction['away_xg'],
        'confidence': prediction.get('confidence_score', 0.85),
        'created_at': datetime.now().isoformat()
    }

    if value_bets and len(value_bets) > 0:
        best = value_bets[0]
        prediction_data['best_bet_market'] = best['market']
        prediction_data['best_bet_selection'] = best['selection']
        prediction_data['best_bet_odds'] = best['odds']
        prediction_data['best_bet_ev'] = best['ev']
        prediction_data['best_bet_kelly'] = best['kelly_percentage']

    result = self.supabase.table('predictions').insert(prediction_data).execute()

    # Get the prediction ID
    if result.data and len(result.data) > 0:
        prediction_id = result.data[0]['id']

        # Save value bets
        if value_bets:
            for bet in value_bets:
                bet_data = {
                    'prediction_id': prediction_id,
                    'home_team': prediction['home_team'],
                    'away_team': prediction['away_team'],
                    'market': bet['market'],
                    'selection': bet['selection'],
                    'odds': bet['odds'],
                    'our_probability': bet['our_probability'],
                    'ev': bet['ev'],
                    'kelly_percentage': bet['kelly_percentage'],
                    'recommended_stake_percentage': bet['recommended_stake_percentage'],
                    'created_at': datetime.now().isoformat()
```

```
                }
                self.supabase.table('value_bets').insert(bet_data).execute()

        logger.info(f"Saved prediction for {prediction['home_team']} vs
{prediction['away_team']}")
        return True

    except Exception as e:
        logger.error(f"Failed to save prediction: {e}")
        return False

def get_recent_predictions(self, limit: int = 10) -> List[Dict]:
    """Get recent predictions from database"""
    if not self.supabase:
        return []

    try:
        result = self.supabase.table('predictions')\
            .select('*')\
            .order('created_at', desc=True)\
            .limit(limit)\
            .execute()

        return result.data
    except Exception as e:
        logger.error(f"Failed to fetch predictions: {e}")
        return []

def get_active_value_bets(self, limit: int = 10) -> List[Dict]:
    """Get active value bets from today"""
    if not self.supabase:
        return []

    try:
        # Get bets from last 24 hours
        yesterday = (datetime.now() - timedelta(days=1)).isoformat()

        result = self.supabase.table('value_bets')\
            .select('*')\
            .filter('created_at', 'gte', yesterday)\
            .order('ev', desc=True)\
            .limit(limit)\
            .execute()
```

```
        return result.data
    except Exception as e:
        logger.error(f"Failed to fetch value bets: {e}")
        return []


# Test function
if __name__ == "__main__":
    # Initialize predictor
    predictor = FootyEdgePredictor()

    # Test prediction
    print("Testing FootyEdge AI Predictor...")
    print("-" * 50)

    prediction = predictor.predict_match("Manchester City", "Arsenal")

    print(f"\nMatch: {prediction['home_team']} vs {prediction['away_team']}")
    print(f"Prediction:")
    print(f"  {prediction['home_team']} Win: {prediction['home_win']:.1%}")
    print(f"  Draw: {prediction['draw']:.1%}")
    print(f"  {prediction['away_team']} Win: {prediction['away_win']:.1%}")
    print(f"\nExpected Goals:")
    print(f"  {prediction['home_team']}: {prediction['home_xg']:.2f}")
    print(f"  {prediction['away_team']}: {prediction['away_xg']:.2f}")
    print(f"\nConfidence: {prediction['confidence_score']:.1%}")

    # Test value bets
    mock_odds = {'home_win': 1.85, 'draw': 3.40, 'away_win': 4.20}
    value_bets = predictor.find_value_bets(prediction, mock_odds)

    if value_bets:
        print(f"\nValue Bets Found:")
        for bet in value_bets:
            print(f"  {bet['selection']} @ {bet['odds']:.2f} (EV: +{bet['ev']:.1%})")
            print(f"    Kelly: {bet['kelly_percentage']:.1%} | Stake:
{bet['recommended_stake_percentage']:.1%}")

    print("\n✅ Test complete!")
```

File 2: api.py (FastAPI Backend)

```python
"""
FootyEdge AI - REST API
FastAPI backend with prediction endpoints
"""

from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field
from typing import Optional, List
import os
import logging
from datetime import datetime

from predictor import FootyEdgePredictor

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Initialize FastAPI
app = FastAPI(
    title="FootyEdge AI API",
    description="Zero-cost football betting intelligence platform",
    version="1.0.0",
    docs_url="/docs",
    redoc_url="/redoc"
)

# CORS middleware - allow all origins for development
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Initialize predictor
predictor = FootyEdgePredictor()

# ==========================================
```

```
# Pydantic Models
# ==============================================

class PredictRequest(BaseModel):
    """Request model for prediction endpoint"""
    home_team: str = Field(..., description="Home team name", example="Manchester City")
    away_team: str = Field(..., description="Away team name", example="Arsenal")

    class Config:
        json_schema_extra = {
            "example": {
                "home_team": "Manchester City",
                "away_team": "Arsenal"
            }
        }

class PredictResponse(BaseModel):
    """Response model for prediction endpoint"""
    home_team: str
    away_team: str
    home_win: float = Field(..., description="Probability of home team winning", ge=0, le=1)
    draw: float = Field(..., description="Probability of draw", ge=0, le=1)
    away_win: float = Field(..., description="Probability of away team winning", ge=0, le=1)
    home_xg: float = Field(..., description="Home team expected goals")
    away_xg: float = Field(..., description="Away team expected goals")
    best_bet: Optional[str] = Field(None, description="Best betting opportunity")
    ev: Optional[float] = Field(None, description="Expected Value of best bet")
    kelly_percentage: Optional[float] = Field(None, description="Kelly Criterion percentage")
    recommended_stake: Optional[str] = Field(None, description="Recommended stake as
percentage")
    confidence_score: float = Field(..., description="Confidence in prediction (0-1)")

    class Config:
        json_schema_extra = {
            "example": {
                "home_team": "Manchester City",
                "away_team": "Arsenal",
                "home_win": 0.52,
                "draw": 0.25,
                "away_win": 0.23,
                "home_xg": 1.85,
                "away_xg": 1.20,
                "best_bet": "Manchester City to win @ 1.85",
                "ev": 0.123,
```

```
            "kelly_percentage": 0.08,
            "recommended_stake": "2.0% of bankroll",
            "confidence_score": 0.85
        }
    }

class ValueBetResponse(BaseModel):
    """Response model for value bets endpoint"""
    match: str
    market: str
    selection: str
    odds: float
    our_probability: float
    implied_probability: float
    ev: float
    kelly_percentage: float
    recommended_stake_percentage: float
    recommendation: str

class TeamStatsResponse(BaseModel):
    """Response model for team stats endpoint"""
    team_name: str
    elo_rating: float
    form_rating: float
    attack_strength: float
    defense_strength: float
    home_advantage: float
    last_5_results: List[str]

# ============================================
# Health Check Endpoints
# ============================================

@app.get("/")
async def root():
    """Root endpoint with system information"""
    return {
        "name": "FootyEdge AI",
        "version": "1.0.0",
        "status": "running",
        "cost": "$0/month",
        "features": [
            "match_prediction",
            "value_bet_detection",
```

```
            "kelly_criterion",
            "telegram_bot",
            "realtime_updates"
        ],
        "endpoints": {
            "predict": "POST /predict",
            "predictions": "GET /predictions",
            "value_bets": "GET /value-bets",
            "teams": "GET /teams/{team_name}",
            "docs": "/docs"
        },
        "timestamp": datetime.now().isoformat()
    }

@app.get("/health")
async def health_check():
    """Health check endpoint for monitoring"""
    return {
        "status": "healthy",
        "database": "connected" if predictor.supabase else "disconnected",
        "timestamp": datetime.now().isoformat()
    }

# ============================================
# Prediction Endpoints
# ============================================

@app.post("/predict", response_model=PredictResponse)
async def predict_match(request: PredictRequest):
    """
    Get match prediction with probabilities, xG, and value bets

    - **home_team**: Name of the home team
    - **away_team**: Name of the away team

    Returns:
    - Win/draw/loss probabilities
    - Expected goals (xG)
    - Best value bet if found
    - Kelly Criterion stake recommendation
    """
    try:
        # Validate input
        if not request.home_team or not request.away_team:
```

```
    raise HTTPException(status_code=400, detail="Team names cannot be empty")

# Get prediction
prediction = predictor.predict_match(request.home_team, request.away_team)

# Mock odds (in production, fetch from live odds API)
mock_odds = {
    'home_win': 1.85,
    'draw': 3.40,
    'away_win': 4.20
}

# Find value bets
value_bets = predictor.find_value_bets(prediction, mock_odds)

# Prepare response
best_bet_text = None
best_ev = None
best_kelly = None
stake_text = None

if value_bets:
    best = value_bets[0]
    best_bet_text = f"{best['selection']} @ {best['odds']:.2f}"
    best_ev = best['ev']
    best_kelly = best['kelly_percentage']
    stake_text = f"{best['recommended_stake_percentage']:.1%} of bankroll"

# Save to database (async in production)
predictor.save_prediction(prediction, value_bets)

return PredictResponse(
    home_team=prediction['home_team'],
    away_team=prediction['away_team'],
    home_win=prediction['home_win'],
    draw=prediction['draw'],
    away_win=prediction['away_win'],
    home_xg=prediction['home_xg'],
    away_xg=prediction['away_xg'],
    best_bet=best_bet_text,
    ev=best_ev,
    kelly_percentage=best_kelly,
    recommended_stake=stake_text,
    confidence_score=prediction['confidence_score']
```

```
        )

    except Exception as e:
        logger.error(f"Prediction error: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/predictions")
async def get_predictions(limit: int = Query(10, ge=1, le=50)):
    """
    Get recent predictions from database

    - **limit**: Number of predictions to return (max 50)
    """
    try:
        predictions = predictor.get_recent_predictions(limit)
        return {
            "predictions": predictions,
            "total": len(predictions),
            "timestamp": datetime.now().isoformat()
        }
    except Exception as e:
        logger.error(f"Error fetching predictions: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/value-bets")
async def get_value_bets(limit: int = Query(10, ge=1, le=50)):
    """
    Get active value bets

    - **limit**: Number of value bets to return (max 50)
    """
    try:
        bets = predictor.get_active_value_bets(limit)

        # Format response
        formatted_bets = []
        for bet in bets:
            formatted_bets.append({
                "match": f"{bet.get('home_team', '')} vs {bet.get('away_team', '')}",
                "market": bet.get('market', ''),
                "selection": bet.get('selection', ''),
                "odds": bet.get('odds', 0),
                "our_probability": bet.get('our_probability', 0),
                "implied_probability": 1 / bet.get('odds', 1) if bet.get('odds') else 0,
```

```
            "ev": bet.get('ev', 0),
            "kelly_percentage": bet.get('kelly_percentage', 0),
            "recommended_stake_percentage": bet.get('recommended_stake_percentage', 0),
            "recommendation": "Strong Buy" if bet.get('ev', 0) > 0.15 else "Buy" if bet.get('ev', 0) >
0.08 else "Consider"
        })

    return {
        "value_bets": formatted_bets,
        "total": len(formatted_bets),
        "timestamp": datetime.now().isoformat()
    }
  except Exception as e:
    logger.error(f"Error fetching value bets: {e}")
    raise HTTPException(status_code=500, detail=str(e))

@app.get("/teams/{team_name}", response_model=TeamStatsResponse)
async def get_team_stats(team_name: str):
  """
  Get detailed statistics for a team

  - **team_name**: Name of the team
  """
  try:
    # Get team matches
    matches = predictor.get_team_matches(team_name, limit=10)

    # Calculate stats
    elo_rating = predictor.calculate_elo_rating(team_name, matches)
    form_rating = predictor.calculate_form_rating(matches)
    attack_strength = predictor.calculate_attack_strength(matches)
    defense_strength = predictor.calculate_defense_strength(matches)

    # Get last 5 results
    last_5_results = [m['result'][0].upper() for m in matches[:5]]

    return TeamStatsResponse(
        team_name=team_name,
        elo_rating=round(elo_rating),
        form_rating=round(form_rating, 2),
        attack_strength=round(attack_strength, 2),
        defense_strength=round(defense_strength, 2),
        home_advantage=50,
        last_5_results=last_5_results
```

```
        )

    except Exception as e:
        logger.error(f"Error fetching team stats: {e}")
        raise HTTPException(status_code=500, detail=str(e))

# ==============================================
# Batch Prediction Endpoint
# ==============================================

class BatchPredictRequest(BaseModel):
    """Request model for batch prediction"""
    matches: List[PredictRequest]

@app.post("/predict/batch")
async def predict_batch(request: BatchPredictRequest):
    """
    Get predictions for multiple matches in one request

    - **matches**: List of matches to predict
    """
    try:
        results = []
        for match in request.matches:
            prediction = predictor.predict_match(match.home_team, match.away_team)
            results.append(prediction)

        return {
            "results": results,
            "total": len(results),
            "timestamp": datetime.now().isoformat()
        }
    except Exception as e:
        logger.error(f"Batch prediction error: {e}")
        raise HTTPException(status_code=500, detail=str(e))

# ==============================================
# Error Handlers
# ==============================================

@app.exception_handler(HTTPException)
async def http_exception_handler(request, exc):
    return JSONResponse(
        status_code=exc.status_code,
```

```
        content={
            "error": exc.detail,
            "status_code": exc.status_code,
            "timestamp": datetime.now().isoformat()
        }
    )

@app.exception_handler(Exception)
async def general_exception_handler(request, exc):
    logger.error(f"Unhandled exception: {exc}")
    return JSONResponse(
        status_code=500,
        content={
            "error": "Internal server error",
            "status_code": 500,
            "timestamp": datetime.now().isoformat()
        }
    )

# ==============================================
# Run with: uvicorn api:app --reload
# ==============================================

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

File 3: bot.py (Telegram Bot)

```python
"""
FootyEdge AI - Telegram Bot
Conversational interface for match predictions
"""

import os
import logging
import requests
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import Application, CommandHandler, CallbackQueryHandler, ContextTypes
from telegram.constants import ParseMode
```

```
from datetime import datetime

# Configure logging
logging.basicConfig(
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    level=logging.INFO
)
logger = logging.getLogger(__name__)

# Configuration
TELEGRAM_TOKEN = os.environ.get('TELEGRAM_BOT_TOKEN')
API_URL = os.environ.get('API_URL', 'https://footyedge-api.onrender.com')

# Bot commands
COMMANDS = {
    'start': 'Welcome to FootyEdge AI',
    'predict': 'Predict match: /predict Man City vs Arsenal',
    'value': 'Get today\'s value bets',
    'help': 'Show help message',
    'about': 'About FootyEdge AI'
}

# ============================================
# Command Handlers
# ============================================

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Send welcome message on /start"""
    user = update.effective_user
    welcome_message = f"""
⚽ *Welcome to FootyEdge AI, {user.first_name}!*

I'm your AI-powered football betting assistant. I predict match outcomes, find value bets, and
help you make smarter betting decisions.

🔮 *What I can do:*
• Predict match winners with probabilities
• Calculate Expected Goals (xG)
• Find value bets with positive EV
• Apply Kelly Criterion for optimal stakes
• Track your betting performance

📱 *How to use me:*
• `/predict Man City vs Arsenal` - Get match prediction
```

```
• `/value` - See today's best value bets
• `/help` - Show all commands

💰 *Built on zero-cost infrastructure*
🆓 *100% free to use*

Ready to win? Let's go! 🚀
"""
    await update.message.reply_text(
        welcome_message,
        parse_mode=ParseMode.MARKDOWN
    )

async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Send help message"""
    help_text = """
📖 *FootyEdge AI Help*

*Available Commands:*

/predict [Team1] vs [Team2]
    Get detailed match prediction with probabilities and xG
    Example: `/predict Manchester City vs Arsenal`

/value
    Show today's best value bets with EV and Kelly recommendations

/start
    Welcome message and introduction

/help
    Show this help message

/about
    About FootyEdge AI and how it works

*Example Prediction Response:*
```

⚽ Manchester City vs Arsenal

📊 Prediction:
🏠 Man City: 52%
🤝 Draw: 25%
```
```

✈️ Arsenal: 23%

🎯 Expected Goals:
🏠 Man City: 1.85
✈️ Arsenal: 1.20

💰 Value Bet Found!
Man City to win @ 1.85
EV: +12.3% | Stake: 2.0%

```

*Tips:*
• Use full team names for best results
• Check /value daily for best opportunities
• Always practice responsible betting

*Data Sources:* Football-Data.org, statistical models
*Cost:* Free forever!
"""
    await update.message.reply_text(
        help_text,
        parse_mode=ParseMode.MARKDOWN
    )

async def about(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Send about information"""
    about_text = """
🤖 *About FootyEdge AI*

FootyEdge AI is an enterprise-grade football betting intelligence platform built entirely on free infrastructure.

*Technology Stack:*
• FastAPI backend (Python)
• Supabase PostgreSQL database
• Telegram Bot API
• Football-Data.org API
• Statistical models (Elo, Poisson, Kelly)

*How It Works:*
1. Fetches historical match data from free APIs
2. Calculates team Elo ratings with recency weighting
3. Models goal distributions using Poisson
```

4. Finds value bets using Expected Value (EV)
5. Optimizes stakes with Kelly Criterion

Key Features:
• Win/draw/loss probabilities
• Expected Goals (xG) calculation
• Value bet detection (EV > 5%)
• Kelly Criterion bet sizing
• Real-time predictions
• Zero cost to use

Why Free?
We believe in democratizing sports analytics. Everyone deserves access to professional-grade betting intelligence.

Responsible Betting:
Remember that betting involves risk. Only bet what you can afford to lose. Use FootyEdge AI as a tool, not a guarantee.

Created with ❤️ for the football community
"""
    await update.message.reply_text(
        about_text,
        parse_mode=ParseMode.MARKDOWN
)

```
async def predict(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Handle /predict command"""
    if not context.args:
        await update.message.reply_text(
            "Please specify teams!\n"
            "Example: `/predict Manchester City vs Arsenal`",
            parse_mode=ParseMode.MARKDOWN
        )
        return

    # Parse teams from command
    text = ' '.join(context.args)
    if 'vs' not in text.lower():
        await update.message.reply_text(
            "Use format: Team1 vs Team2\n"
            "Example: `/predict Manchester City vs Arsenal`",
            parse_mode=ParseMode.MARKDOWN
        )
```

```
        return

    parts = text.split('vs')
    home_team = parts[0].strip()
    away_team = parts[1].strip()

    # Show typing indicator
    await update.message.chat.send_action(action="typing")

    try:
        # Call the API
        response = requests.post(
            f"{API_URL}/predict",
            json={'home_team': home_team, 'away_team': away_team},
            timeout=15
        )

        if response.status_code == 200:
            data = response.json()

            # Format prediction message
            message = self._format_prediction_message(data)
            await update.message.reply_text(
                message,
                parse_mode=ParseMode.MARKDOWN
            )

            # Add inline buttons for more actions
            keyboard = [
                [
                    InlineKeyboardButton("📊 More Stats",
callback_data=f"stats_{home_team}_{away_team}"),
                    InlineKeyboardButton("💰 Value Bets",
callback_data=f"value_{home_team}_{away_team}")
                ]
            ]
            reply_markup = InlineKeyboardMarkup(keyboard)
            await update.message.reply_text(
                "What would you like to do next?",
                reply_markup=reply_markup
            )

        elif response.status_code == 404:
            await update.message.reply_text(
```

```
            "❌ Team not found. Please check team names and try again.\n"
            "Example: `Manchester City vs Arsenal`",
            parse_mode=ParseMode.MARKDOWN
        )
    else:
        await update.message.reply_text(
            "❌ Error getting prediction. Please try again later."
        )

except requests.exceptions.Timeout:
    await update.message.reply_text(
        "⏰ Request timed out. The API might be starting up. Please try again in a few
seconds."
    )
except Exception as e:
    logger.error(f"Prediction error: {e}")
    await update.message.reply_text(
        f"❌ Error: {str(e)}\nPlease try again later."
    )

async def value_bets(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Handle /value command - show value bets"""
    await update.message.chat.send_action(action="typing")

    try:
        response = requests.get(f"{API_URL}/value-bets", timeout=10)

        if response.status_code == 200:
            data = response.json()
            value_bets = data.get('value_bets', [])

            if not value_bets:
                await update.message.reply_text(
                    "📭 No value bets found for today.\n"
                    "Check back later or try different matches!",
                    parse_mode=ParseMode.MARKDOWN
                )
                return

            message = self._format_value_bets_message(value_bets)
            await update.message.reply_text(
                message,
                parse_mode=ParseMode.MARKDOWN
            )
```

```
        else:
            await update.message.reply_text(
                "❌ Error fetching value bets. Please try again later."
            )

    except Exception as e:
        logger.error(f"Value bets error: {e}")
        await update.message.reply_text(
            f"❌ Error: {str(e)}\nPlease try again later."
        )

async def button_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Handle inline button callbacks"""
    query = update.callback_query
    await query.answer()

    data = query.data
    parts = data.split('_')

    if parts[0] == 'stats':
        # Get team stats
        home = parts[1]
        away = parts[2]

        try:
            # Get both team stats
            home_response = requests.get(f"{API_URL}/teams/{home}", timeout=10)
            away_response = requests.get(f"{API_URL}/teams/{away}", timeout=10)

            if home_response.status_code == 200 and away_response.status_code == 200:
                home_stats = home_response.json()
                away_stats = away_response.json()

                stats_message = self._format_stats_message(home_stats, away_stats)
                await query.edit_message_text(
                    stats_message,
                    parse_mode=ParseMode.MARKDOWN
                )
            else:
                await query.edit_message_text("❌ Could not fetch team stats.")

        except Exception as e:
            logger.error(f"Stats fetch error: {e}")
```

```
            await query.edit_message_text(f"❌ Error: {str(e)}")

    elif parts[0] == 'value':
        # Show value bets for specific match
        home = parts[1]
        away = parts[2]

        await query.edit_message_text(
            f"🔍 Fetching value bets for {home} vs {away}...\n\n"
            f"Use `/value` to see all value bets for today.",
            parse_mode=ParseMode.MARKDOWN
        )

# ============================================
# Message Formatting Helpers
# ============================================

def _format_prediction_message(self, data: dict) -> str:
    """Format prediction data into a nice message"""

    # Create progress bar for probabilities
    def prob_bar(prob):
        filled = int(prob * 10)
        return "▓" * filled + "▒" * (10 - filled)

    message = f"""
⚽ *{data['home_team']} vs {data['away_team']}*

📊 *Prediction:*
🏠 *{data['home_team']}*: {data['home_win']:.1%} {prob_bar(data['home_win'])}
🤝 *Draw*: {data['draw']:.1%} {prob_bar(data['draw'])}
✈️ *{data['away_team']}*: {data['away_win']:.1%} {prob_bar(data['away_win'])}

🎯 *Expected Goals:*
🏠 {data['home_team']}: {data['home_xg']:.2f}
✈️ {data['away_team']}: {data['away_xg']:.2f}

📈 *Confidence:* {data['confidence_score']:.1%}
"""

    if data.get('best_bet'):
        message += f"""
💰 *Value Bet Found!*
{data['best_bet']}
```

```
EV: +{data['ev']:.1%} | Kelly: {data['kelly_percentage']:.1%}
Recommended Stake: {data['recommended_stake']}
"""
    else:
        message += """
⚠️ *No Value Bets Found*
Current odds don't offer positive expected value.
"""

    message += """
---
_Data: Statistical models + Free APIs_
_Always bet responsibly_
"""

    return message

def _format_value_bets_message(self, value_bets: list) -> str:
    """Format value bets into a nice message"""

    message = "💰 *Today's Value Bets*\n\n"

    for i, bet in enumerate(value_bets[:5], 1):
        # Emoji based on EV
        if bet['ev'] > 0.15:
            emoji = "🔥"
        elif bet['ev'] > 0.08:
            emoji = "✅"
        else:
            emoji = "📈"

        message += f"{emoji} *{i}. {bet['match']}*\n"
        message += f"   • {bet['selection']} @ {bet['odds']:.2f}\n"
        message += f"   • EV: +{bet['ev']:.1%} | Kelly: {bet['kelly_percentage']:.1%}\n"
        message += f"   • Stake: {bet['recommended_stake_percentage']:.1%} of bankroll\n"
        message += f"   • *Recommendation:* {bet['recommendation']}\n\n"

    message += """
---
💡 *How to Use:*
1. Open your betting app
2. Place bets with recommended stakes
3. Use Kelly sizing for optimal growth
4. Track your results
```

```
⚠️ *Remember:* Only bet what you can afford to lose!
"""

    return message

def _format_stats_message(self, home_stats: dict, away_stats: dict) -> str:
    """Format team stats comparison"""

    message = f"""
📊 *Team Stats Comparison*

*{home_stats['team_name']}* vs *{away_stats['team_name']}*

| Category | {home_stats['team_name'][:10]} | {away_stats['team_name'][:10]} |
|----------|------|------|
| Elo Rating | {home_stats['elo_rating']} | {away_stats['elo_rating']} |
| Form (0-1) | {home_stats['form_rating']:.2f} | {away_stats['form_rating']:.2f} |
| Attack | {home_stats['attack_strength']:.2f} | {away_stats['attack_strength']:.2f} |
| Defense | {home_stats['defense_strength']:.2f} | {away_stats['defense_strength']:.2f} |

*Recent Form:* {''.join(home_stats['last_5_results'])} vs {''.join(away_stats['last_5_results'])}

*Analysis:*
{'🏠 Home advantage + home team stronger' if home_stats['elo_rating'] >
away_stats['elo_rating'] else '✈️ Away team has better form'}
"""

    return message

# ==========================================
# Main Function
# ==========================================

def main():
    """Start the bot"""
    if not TELEGRAM_TOKEN:
        logger.error("TELEGRAM_BOT_TOKEN not set!")
        return

    # Create application
    application = Application.builder().token(TELEGRAM_TOKEN).build()

    # Add command handlers
```

```
    application.add_handler(CommandHandler("start", start))
    application.add_handler(CommandHandler("help", help_command))
    application.add_handler(CommandHandler("about", about))
    application.add_handler(CommandHandler("predict", predict))
    application.add_handler(CommandHandler("value", value_bets))

    # Add callback handler for inline buttons
    application.add_handler(CallbackQueryHandler(button_callback))

    # Start bot
    logger.info("🤖 FootyEdge AI Bot is running...")
    application.run_polling(allowed_updates=Update.ALL_TYPES)

if __name__ == "__main__":
    main()
```

Step-by-Step Instructions for AI Studio

1. Create the PDF Files

Save each of the above sections as separate PDFs, or combine into one master PDF. Use any PDF creator (Google Docs → Export as PDF works great).

2. Upload to AI Studio

1. Go to https://aistudio.google.com
2. Click "Create New" → "Upload Files"
3. Upload your PDF(s)
4. In the prompt box, paste:

```
I've uploaded the complete FootyEdge AI technical specification. Based on these documents, generate the complete codebase with these files:

1. predictor.py - Full prediction engine with Elo ratings, Poisson distribution, Kelly Criterion
2. api.py - FastAPI backend with all endpoints
3. bot.py - Telegram bot with all commands
4. requirements.txt - All dependencies
5. render.yaml - Render deployment config
6. supabase_schema.sql - Database schema
7. dashboard/App.jsx - React frontend
```

Make sure all code is production-ready, includes error handling, and uses only free services.
```

3. AI Studio Will Generate Complete Code

AI Studio will output all the files with full working code. Copy each one into your repository.

---
```